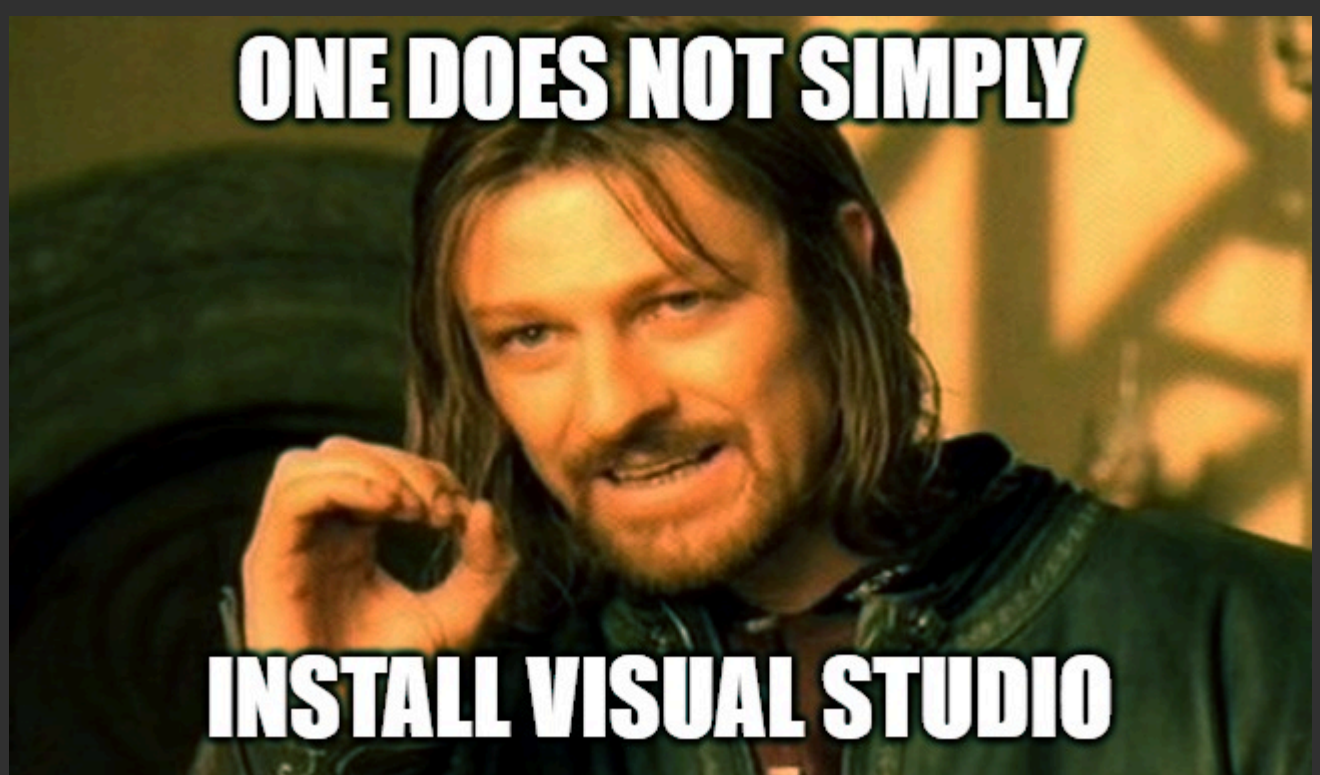
I Fixed Windows Native Development

January 26, 2026 • Jonathan Marler

Imagine you're maintaining a native project. You use Visual Studio for building on Windows, so you do the responsible thing and list it as a dependency

> Build Requirements: Install Visual Studio

If you're lucky enough not to know this yet, I envy you. Unfortunately, at this point even Boromir knows...



Well put Boromir

What you may not realize is, you've actually signed up to be unpaid tech support for Microsoft's "Visual Studio Installer". You might notice GitHub Issues becoming less about your code and more about broken builds, specifically on Windows. You find yourself explaining to a contributor that they didn't check the "Desktop development with C++" workload, but specifically the v143 build tools and the 10.0.22621.0 SDK. No, not that one, the *other* one. You spend less time on your project because you're too busy being a human-powered dependency resolver for a 50GB IDE.

Saying "Install Visual Studio" is like handing contributors a choose-your-own-adventure book riddled with bad endings, some of which don't let you go back. I've had to re-image my entire OS more than once over the years.

Why is this tragedy unique to Windows?

On **Linux**, the toolchain is usually just a package manager command away. On the other hand, “Visual Studio” is thousands of components. It’s so vast that Microsoft distributes it with a sophisticated GUI installer where you navigate a maze of checkboxes, hunting for which “Workloads” or “Individual Components” contain the actual compiler. Select the wrong one and you might lose hours installing something you don’t need. Miss one, like “Windows 10 SDK (10.0.17763.0)” or “Spectre-mitigated libs,” and your build fails three hours later with a cryptic error like `MSB8101`. And heaven help you if you need to downgrade to an older version of the build tools for a legacy project.

The Visual Studio ecosystem is built on a legacy of ‘all-in-one’ monoliths. It conflates the editor, the compiler, and the SDK into a single, tangled web. When we list ‘Visual Studio’ as a dependency, we’re failing to distinguish between the tool we use to write code and the environment required to compile it.

The pain compounds quickly:

- **Hours-long waits**: You spend an afternoon watching a progress bar download 15GB just to get a 50MB compiler.
- **Zero transparency**: You have no idea which files were installed or where they went. Your registry is littered with cruft and background update services are permanent residents of your Task Manager.
- **No version control**: You can’t check your compiler into Git. If a teammate has a slightly different Build Tools version, your builds can silently diverge.
- **The “ghost” environment**: Uninstalling is never truly clean. Moving to a new machine means repeating the entire GUI dance, praying you checked the same boxes.

Even after installation, compiling a single C file from the command line requires finding the Developer Command Prompt. Under the hood, this shortcut invokes `vcvarsall.bat`, a fragile batch script that globally mutates your environment variables just to locate where the compiler is hiding this week.

Ultimately, you end up with build instructions that look like a legal disclaimer:

> “Works on my machine with VS 17.4.2 (Build 33027.167) and SDK 10.0.22621.0. If you have 17.5, please see Issue #412. If you are on ARM64, godspeed.”

On Windows, this has become the “cost of doing business”. We tell users to wait three hours for a 20GB install just so they can compile a 5MB executable. **It’s become an active deterrent to native development**.

A new way

I’m not interested in being a human debugger for someone else’s installer. I want the MSVC toolchain to behave like a modern dependency: versioned, isolated,

declarative.

I spent a few weeks building an open source tool to make things better. It's called msvcup. It's a small CLI program. On good network/hardware, it can install the toolchain/SDK in a few minutes, including everything to cross-compile to/from ARM. Each version of the toolchain/SDK gets its own isolated directory. It's idempotent and fast enough to invoke every time you build. Let's try it out.

Create `hello.c` and `build.bat`:

```
#include <stdio.h>
int main() { printf("Hello, World\n"); }
```

```
@setlocal

@if not exist msvcup.exe (
    echo msvcup.exe: installing...
    curl -L -o msvcup.zip https://github.com/marler8997/msvcup/releases/download/
    tar xf msvcup.zip
    del msvcup.zip
) else (
    echo msvcup.exe: already installed
)
@if not exist msvcup.exe exit /b 1

set MSVC=msvc-14.44.17.14
set SDK=sdk-10.0.22621.7

msvcup install --lock-file msvcup.lock --manifest-update-off %MSVC% %SDK%
@if %errorlevel% neq 0 (exit /b %errorlevel%)

msvcup autoenv --target-cpu x64 --out-dir autoenv %MSVC% %SDK%
@if %errorlevel% neq 0 (exit /b %errorlevel%)

.\autoenv\cl hello.c
```

And we're done.

Believe it or not, this `build.bat` script replaces the need to "Install Visual Studio". This script should run on any Windows system since Windows 10 (assuming it has curl/tar which have been shipped since 2018). It installs the MSVC toolchain, the Windows SDK and then compiles our program.

For my fellow Windows developers, go ahead and take a moment. Visual Studio can't hurt you anymore. The `build.bat` above isn't just a helper script; it's a declaration of independence from the Visual Studio Installer. Our dependencies are fully specified, making builds reproducible across machines. And when those dependencies are installed, they won't pollute your registry or lock you into a single global version.

Also note that after the first run, the `msvcup` commands take milliseconds, meaning we can just leave these commands in our build script and now we have a fully self-

contained script that can build our project on virtually any modern Windows machine.

How?

msvcup is inspired by a small Python script written by Mārtiņš Možeiko. The key insight is that Microsoft publishes JSON manifests describing every component in Visual Studio, the same manifests the official installer uses. msvcup parses these manifests, identifies just the packages needed for compilation (the compiler, linker, headers, and libraries), and downloads them directly from Microsoft's CDN. Everything lands in versioned directories under `C:\msvcup\`. For details on lock files, cross-compilation, and other features, see the msvcup README.md.

The astute will also notice that our `build.bat` script never sources any batch files to set up the "Developer Environment". The script contains two msvcup commands. The first installs the toolchain/SDK, and like a normal installation, it includes "vcvars" scripts to set up a developer environment. Instead, our `build.bat` leverages the `msvcup autoenv` command to create an "Automatic Environment". This creates a directory that contains wrapper executables to set the environment variables on your behalf before forwarding to the underlying tools. It even includes a `toolchain.cmake` file which will point your CMake projects to these tools, allowing you to build your CMake projects outside a special environment.

At Tuple (a pair-programming app), I integrated msvcup into our build system and CI, which allowed us to remove the requirement for the user/CI to pre-install Visual Studio. Tuple compiles hundreds of C/C++ projects including WebRTC. This enabled both x86_64 and ARM builds on the CI as well as keeping the CI and everyone on the same toolchain/SDK.

The benefits:

- **Everything installs into a versioned directory.** No problem installing versions side-by-side. Easy to remove or reinstall if something goes wrong.
- **Cross-compilation enabled out of the box.** msvcup currently always downloads the tools for all supported cross-targets, so you don't have to do any work looking for all the components you need to cross-compile.
- **Lock file support.** A self-contained list of all the payloads/URLs. Everyone uses the same packages, and if Microsoft changes something upstream, you'll know.
- **Blazing fast.** The `install` and `autoenv` commands are idempotent and complete in milliseconds when there's no work to do.

No more "it works on my machine because I have the 2019 Build Tools installed." No more registry-diving to find where `cl.exe` is hiding this week. With msvcup, your environment is defined by your code, portable across machines, and ready to compile in milliseconds.

Limitations

msvcup focuses on the core compilation toolchain. If you need the full Visual Studio IDE you'll still need the official installer. For most native development workflows, though, it covers what you actually need.

A Real-World Example: Building Raylib

Let's try this on a real project. Here's a script that builds raylib from scratch on a clean Windows system. In this case, we'll just use the SDK without the autoenv:

```
@setlocal

set TARGET_CPU=x64

@if not exist msvcup.exe (
    echo msvcup.exe: installing...
    curl -L -o msvcup.zip https://github.com/marler8997/msvcup/releases/download/
    tar xf msvcup.zip
    del msvcup.zip
)

set MSVC=msvc-14.44.17.14
set SDK=sdk-10.0.22621.7

msvcup.exe install --lock-file msvcup.lock --manifest-update-off %MSVC% %SDK%
@if %errorlevel% neq 0 (exit /b %errorlevel%)

@if not exist raylib (
    git clone https://github.com/raysan5/raylib -b 5.5
)

call C:\msvcup\%MSVC%\vcvars-%TARGET_CPU%.bat
call C:\msvcup\%SDK%\vcvars-%TARGET_CPU%.bat

cmd /c "cd raylib\projects\scripts && build-windows"
@if %errorlevel% neq 0 (exit /b %errorlevel%)

@echo build success: game exe at:
@echo .\raylib\projects\scripts\builds\windows-msvc\game.exe
```

No Visual Studio installation. No GUI. No prayer. Just a script that does exactly what it says.

P.S. Here is a page that shows how to use msvcup to build LLVM and Zig from scratch on Windows.

← Creating a Programming Language and God Mode in a Couple weekends

---